Ultrasound Reconstruction via 3D Gaussian Splatting

Technical
University
of Munich

IN2106 - SS2024

Final Presentation

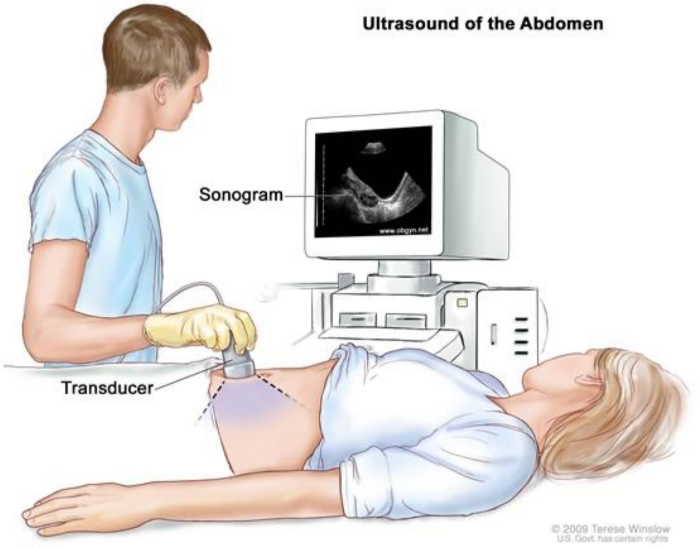
Ahmed Kadri, Florence Klitzner, Xiangning Luo

Tutors: Felix Dülmer, Farid Azampour

Content

- Introduction
- Motivation
- 3D-Gaussian Splatting and NeRF
- Adjusting the Gaussian Splatting Pipeline for Ultrasound
- Sampling-based Renderer
- Projection-based Renderer
- Conclusions

Introduction



Ultrasound [1] (cheap + fast)

MRI [2] (expensive)

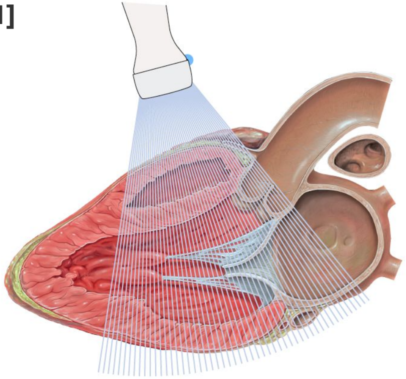
CT [3] (fast+cheap but radiation harmful)

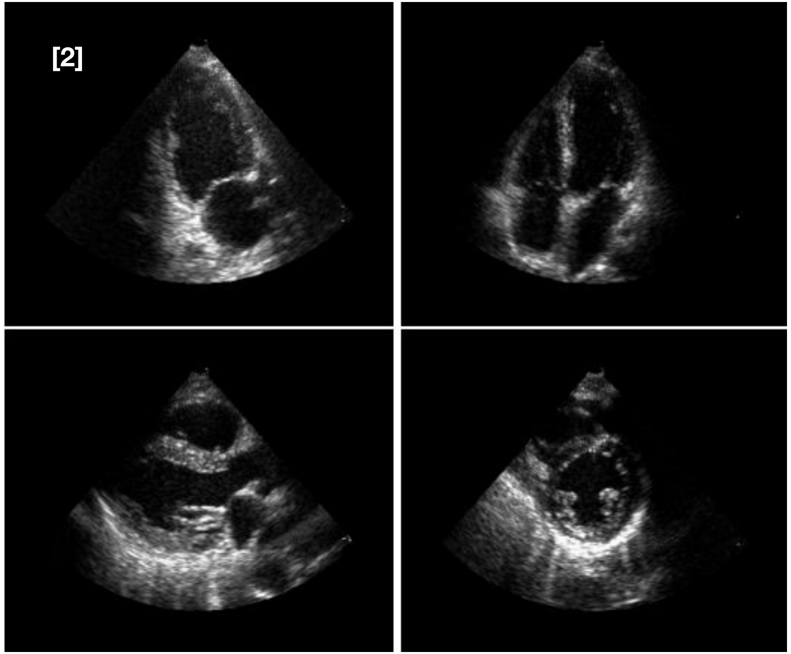
Sources:
[1] https://www.nibib.nih.gov/science-education/science-topics/ultrasound
[2] https://www.mri-roentgen.ch/de/angebot/magnetresonanztomographie-mri/
[3]https://www.touchstoneimaging.com/why-did-my-doctor-order-a-ct-scan-of-my-abdomen-and-pelvis/

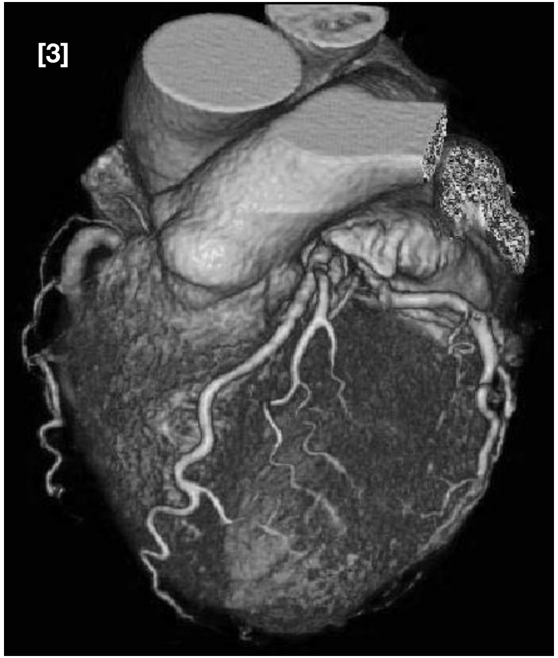
Introduction

2D (two-dimensional) ultrasound

[1]



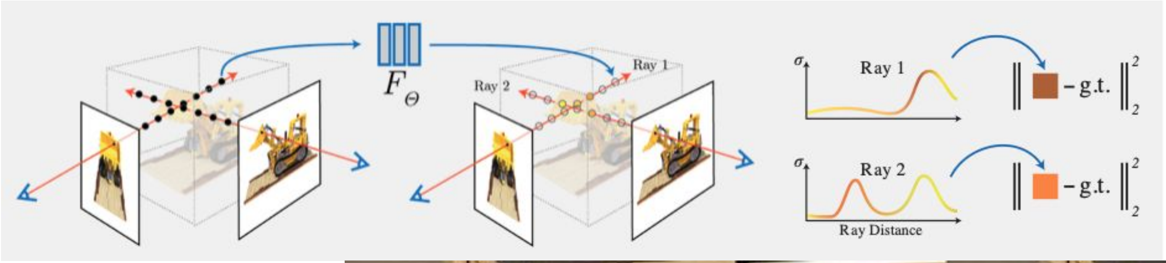


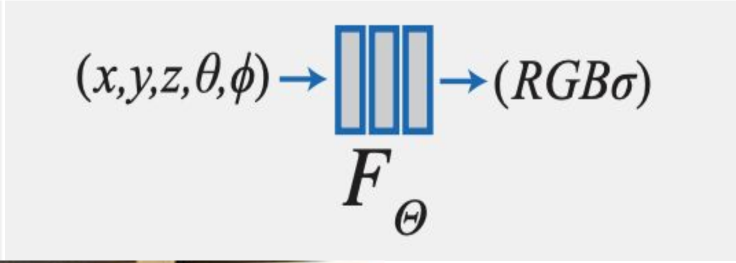


Sources:
[1] https://ecgwaves.com/topic/two-dimensional-2d-echocardiography/
[2]https://www.researchgate.net/figure/Four-different-ultrasound-views-of-the-heart-Clockwise-from-upper-left-apical-two_fig1_228740511
[3]https://www.researchgate.net/figure/Left-Volume-rendering-of-a-human-heart-acquired-using-multi-detector-computed_fig4_224027592

Doctor has to put together the slices to visualize the 3D model in his head

Motivation: Neural Radiance Fields





Sources:
[1] https://www.matthewtancik.com/nerf

Motivation: UltraNeRF

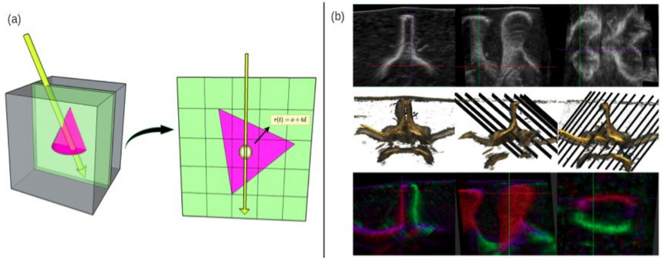


Figure 2: (a) Each ray r corresponds to a single scan-line with origin $\mathbf{o}$ at top of the image plane and direction $\mathbf{d}$ pointing along the scan-line. Query points are defined by their spatial location $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$. (b) Visualization of the phantom dataset: white intensities (top) from max-value compounding of all-angle images; red/blue/green intensities (bottom) compose white intensities by view angle; middle row visualizes observation directions with respect to anatomy.

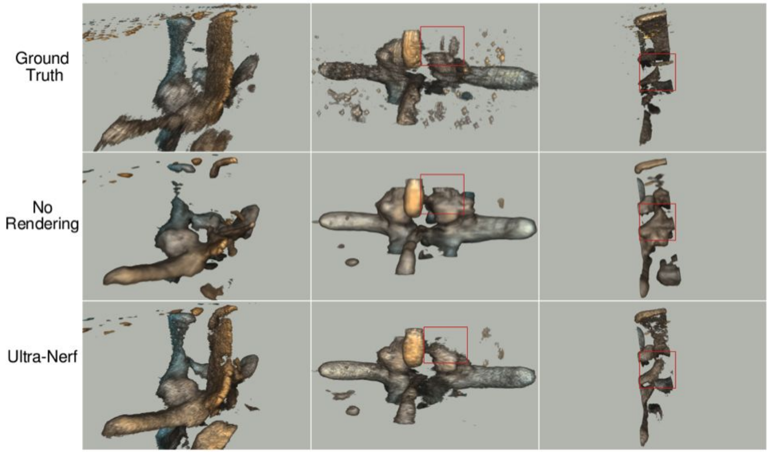


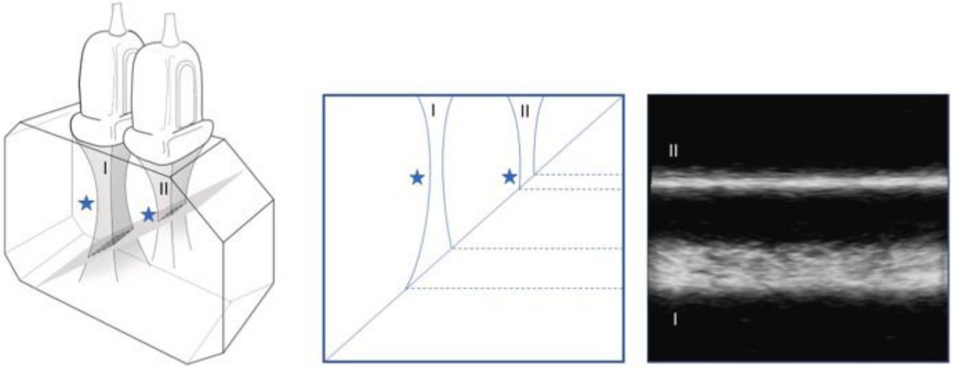
Figure 6: Compounded volumes: without rendering (middle row) the model is not aware of the viewing direction hence occluded parts of the lamina are reconstructed (red). By adding the rendering function, we introduce view-direction dependency needed to reconstruct anisotropic phenomena.

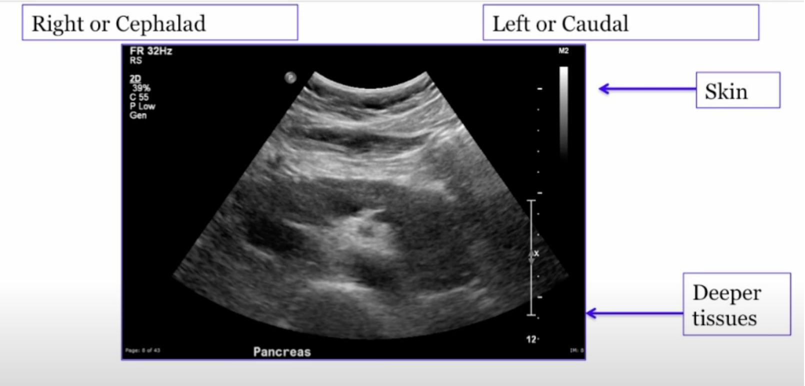
Sources:
[1] Wysocki, M., Azampour, M. F., Eilers, C., Busam, B., Salehi, M., & Navab, N. (2024, January). Ultra-nerf: Neural radiance fields for ultrasound imaging. In Medical Imaging with Deep Learning (pp. 382-401). PMLR.

Motivation: Gaussian Splatting

Sources:
[1] https://youtu.be/9jcl6R7leFE?si=4WjqnWu0jERgVFt6

Ultrasound Imaging





Transmitted → Fluid → No signal = anechoic = dark

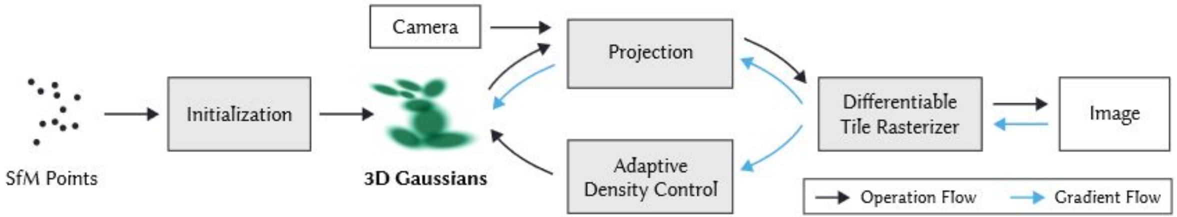
Somewhere in between → Soft tissues / muscles fat → Iso/hypo/hyperechoic = shades of gray

Reflected → Bones, Air → Lots of signal = hyperechoic = bright

Sources:
[1] https://www.youtube.com/watch?v=gT0zV0XYKek

Traditional 3D Gaussian Splatting:

Initialization:
3DGS starts by substituting a sparse point cloud from Structure-from-Motion (SfM) with 3D Gaussians. These represent spatial primitives that can be efficiently rasterized on a GPU for fast rendering.

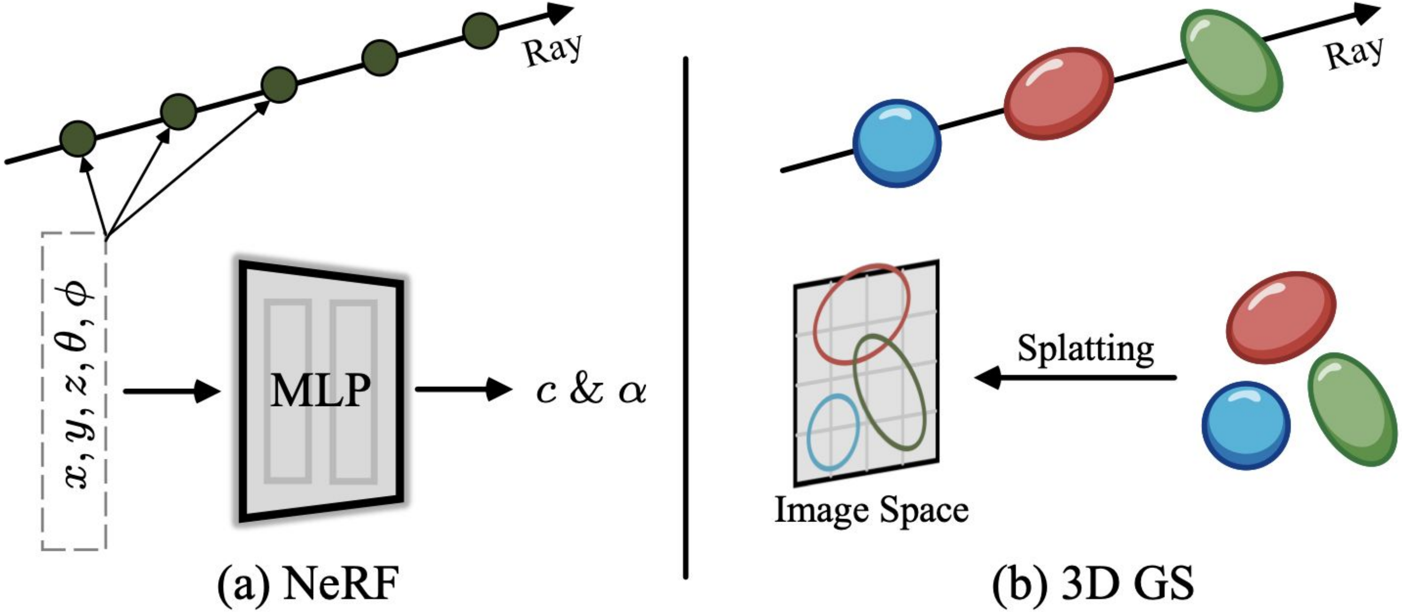


3D-GS Pipeline from [1]

Rendering & Optimization:
3D Gaussians are rendered into 2D images that are compared to ground truth images. The system optimizes the scene by pruning, cloning, or splitting the Gaussians to minimize errors, such as L1-loss and SSIM, improving the quality of the reconstruction.

Sources:
[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Trans. Graph., 42(4), 139-1.

NeRF vs 3D Gaussian Splatting:



Sources:
[1] Chen, G., & Wang, W. (2024). A survey on 3d gaussian splatting. *arXiv preprint arXiv:2401.03890*.

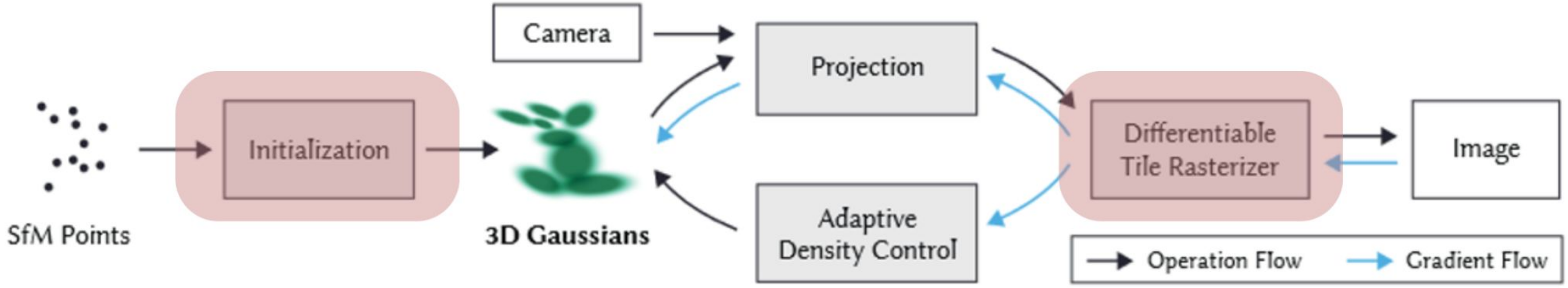
Ultra-NeRF to Ultra-Splatting?

Ultra-NeRF Pipeline:

Ultra-NeRF adapts the NeRF framework for ultrasound imaging, incorporating ray-tracing techniques to handle view-dependent phenomena such as occlusion and acoustic shadows.

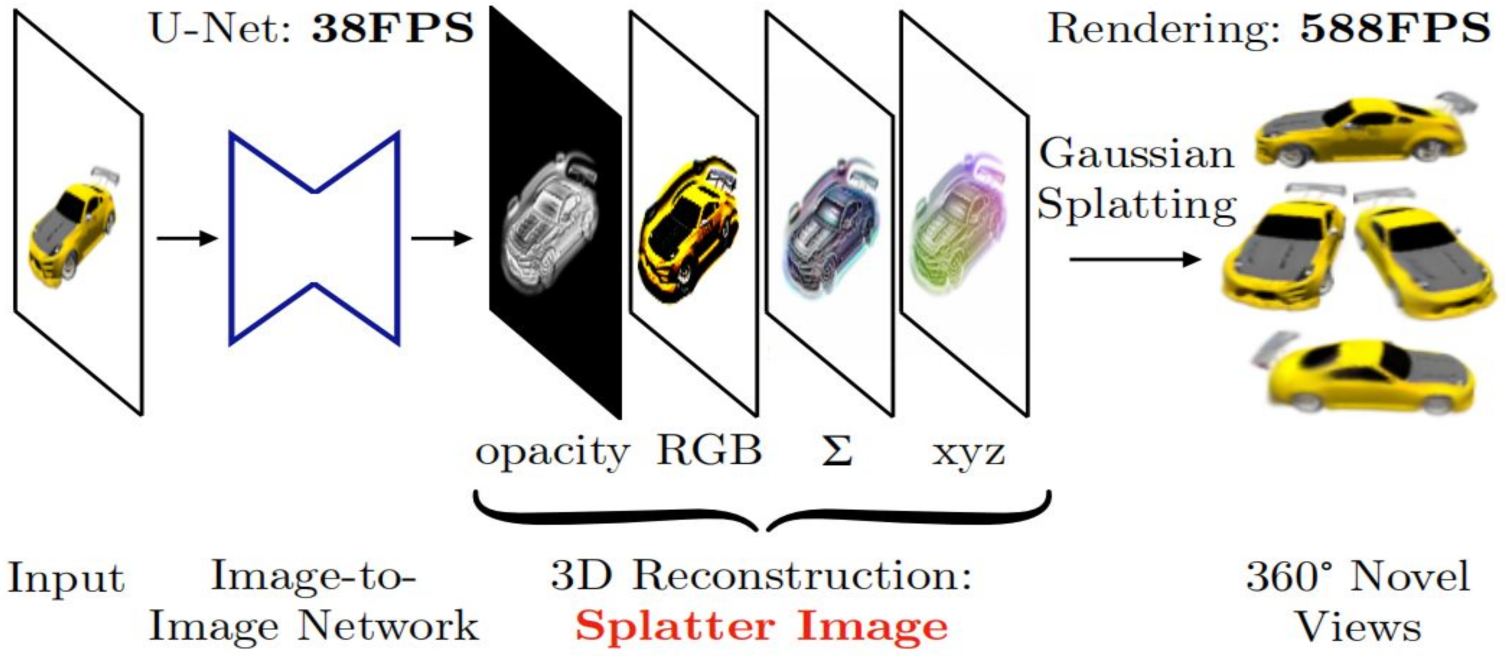
3DGS Adaptation？

Unlike NeRF, 3DGS uses explicit spatial representations. It initializes Gaussians based on geometric data from SfM, making it more efficient and spatially aware in representing complex 3D scenes like medical ultrasound images.



Sources:
[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Trans. Graph., 42(4), 139-1.

Ultra-fast Single-View 3D Reconstruction



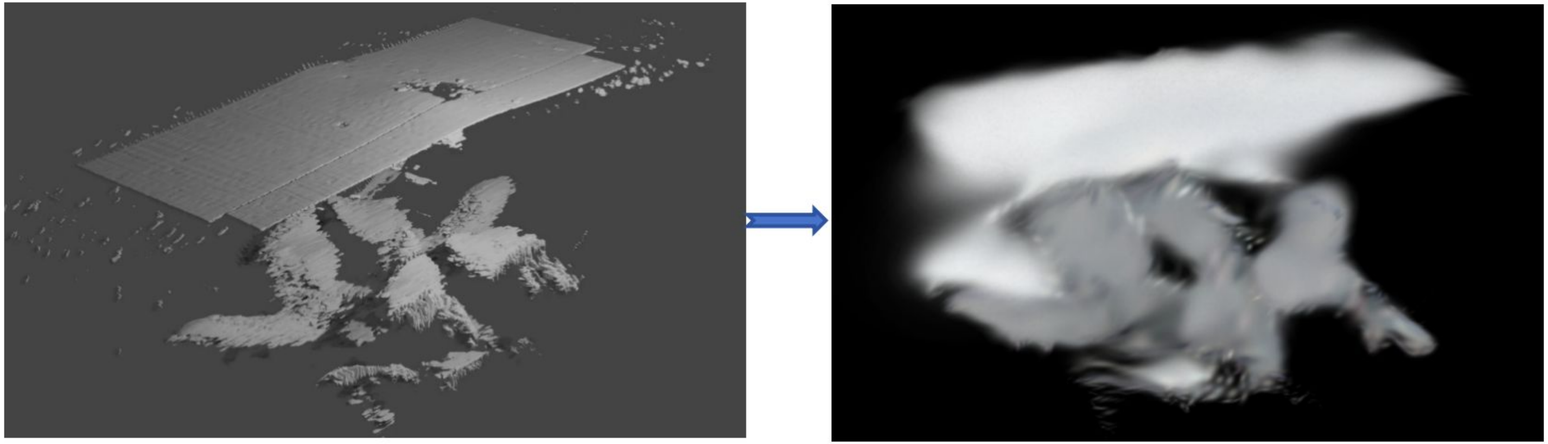
Sources:
[1] Szymanowicz, S., Rupprecht, C., & Vedaldi, A. (2024). Splatter image: Ultra-fast single-view 3d reconstruction. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 10208-10217).

Ultra-fast Single-View 3D Reconstruction

Idea: This method predicts 3D Gaussians for each pixel using an image-to-image neural network, bypassing point cloud reconstruction and allowing for rapid 3D imaging, ideal for real-time applications.

Applicability to B-mode images:

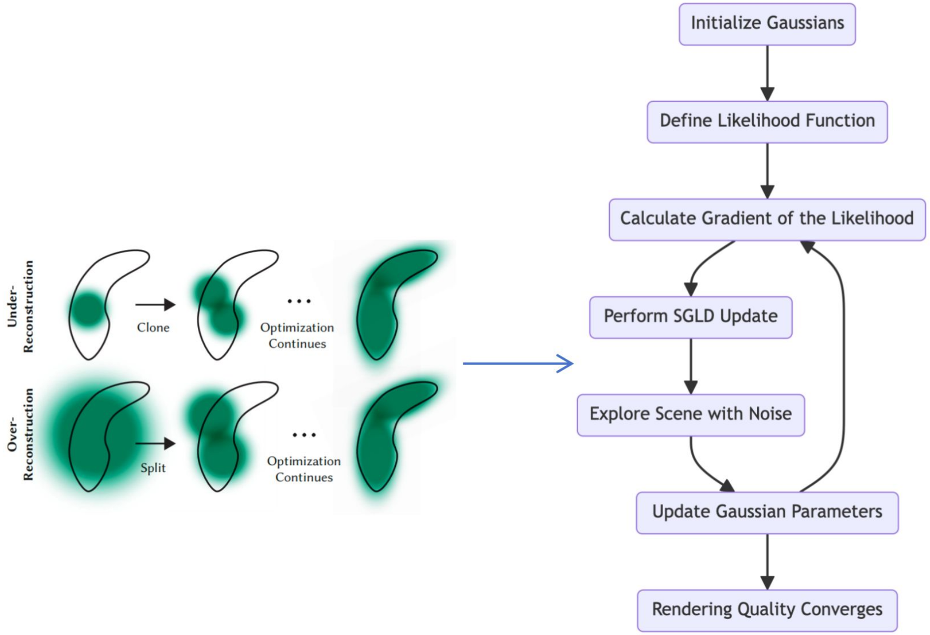
Not practical !



Heuristic free 3DGS (MCMC)

Idea:
By treating 3DGS as a Markov Chain Monte Carlo (MCMC) problem, this approach optimizes Gaussian placement without relying on heuristics, improving rendering quality and computational efficiency.

Applicability to B-mode images:
Meaningful after develop a good reasterizer for B-mode images.



Sources:
[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Trans. Graph., 42(4), 139-1.

SfM Free 3D Gaussian Splatting

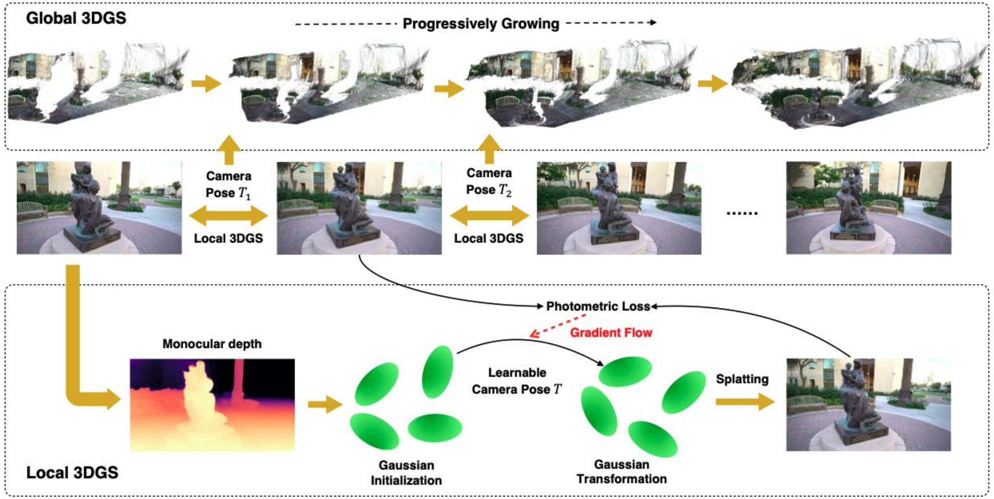
SfM:
- Time-consuming
- Sensitive to feature extraction errors
- Difficulties in handling textureless or repetitive regions

↓

Estimated poses instead of ground-truth poses:
- Temporal continuity from video (small changes between frames)
- Explicit volumetric representation (point cloud)

↓

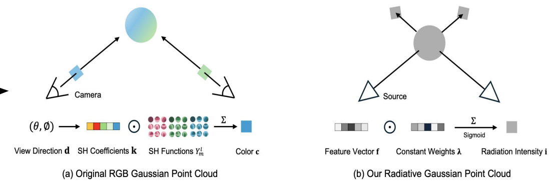
Depth estimation fails on US images which makes the use of the pre-trained neural network to optimize the learned camera pose between frames not feasible.



Source: https://oasisyang.github.io/colmap-free-3dgs/

Applying 3DGS to X-Ray Imaging

Ideas to adapt the method designed for natural light imaging to other imaging modalities
Especially at the level of:
- Initialization
- Rasterization



SH (anisotropic) -> X-Ray (isotropic)

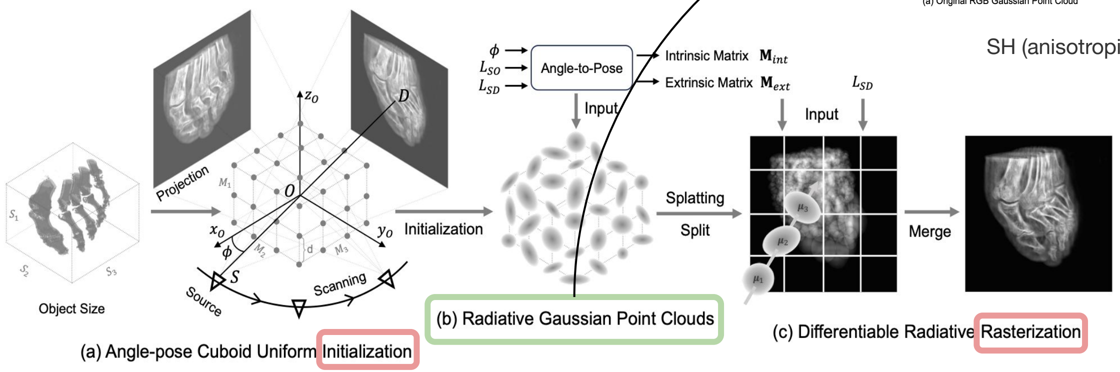
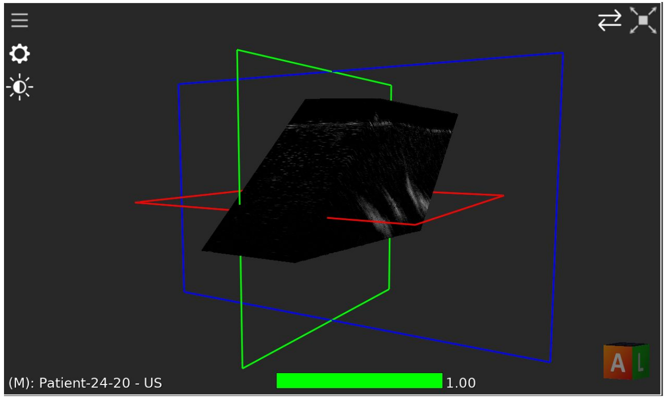
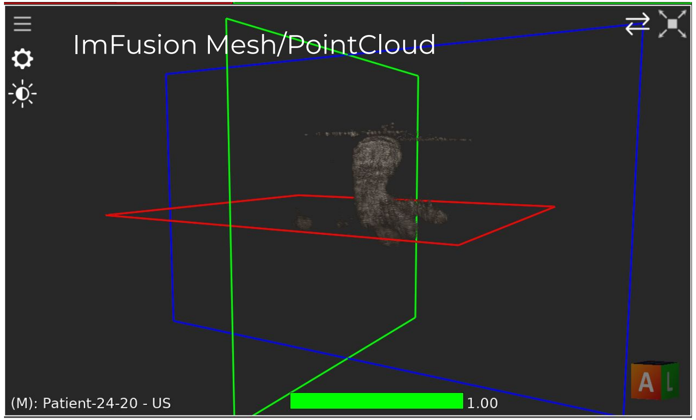


Fig. 3: Pipeline of our method. (a) Angle-pose Cuboid Uniform Initialization (ACUI) strategy uses the parameters of X-ray scanner to compute intrinsic and extrinsic matrices, and samples center points for 3D Gaussians. (b) Our radiative Gaussian point cloud model learns to predict the radiation intensity of 3D points. (c) Based on our Gaussian model, we develop a GPU-friendly Differentiable Radiative Rasterization (DRR).

Sources:
[1] Cai, Y., Liang, Y., Wang, J., Wang, A., Zhang, Y., Yang, X., ... & Yuille, A. (2024). Radiative gaussian splatting for efficient x-ray novel view synthesis. arXiv preprint arXiv:2403.04116.

Initialization of 3D Gaussians

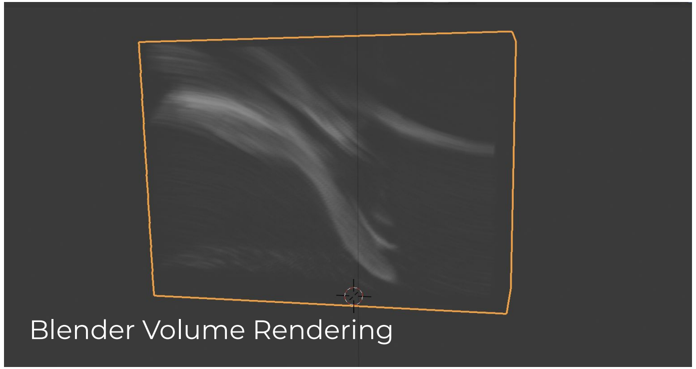




Inspired by X-Gaussian, we decided to replace the sparse point cloud that we get from SfM through:

1. **Mesh**: Place Gaussians at the vertex positions
2. **Volume**: Place Gaussians at the voxels with high intensity values
3. **Otherwise**: Place Gaussians <u>randomly</u>

As for the poses, we have the tracked pose of the robotic arm, which we aim to use in our rendering (projection) step



Apply 3DGS to US images? Ultra-splatting pipeline:

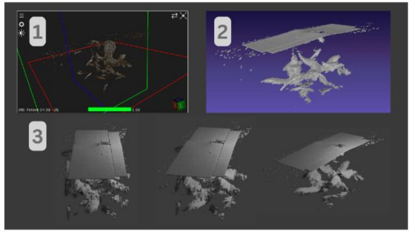


Figure 11: 3DGS Code Understanding Steps

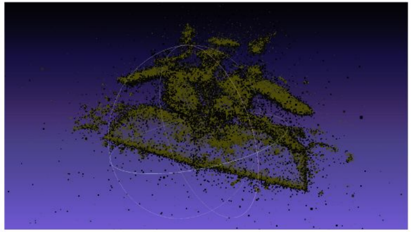


Figure 12: 3DGS Reconstruction Comparison with input, Input (yellow), 3DGS Reconstruction (black)

Import US frames into ImFusion Suite

↓

Apply Volume Compounding to sweeps

↓

Extract mesh from the volume

↓

Import mesh into Blender and record 360 degree video

↓

Split the video into frames

↓

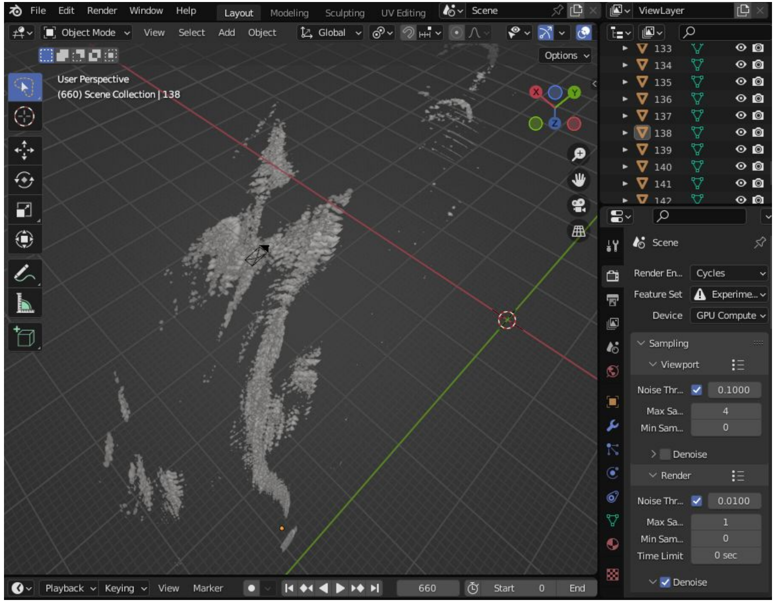
Plug frames into COLMAP (SfM)

↓

Get camera model and sparse pointcloud

↓

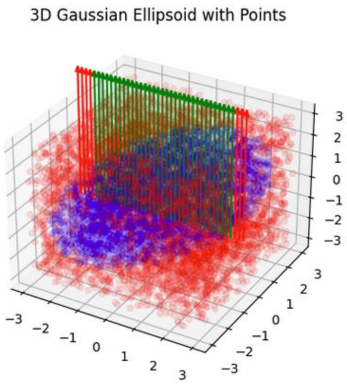
Run 3DGS to see the outcome

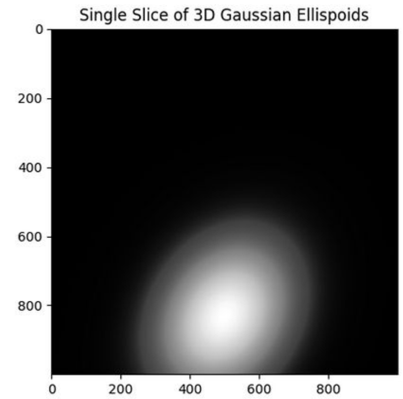


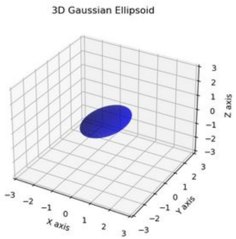
Rendering via Sampling

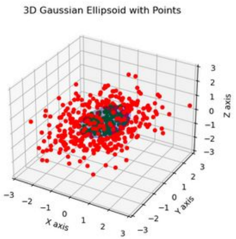
- projection model for gaussians might be physically inaccurate for US image acquisition

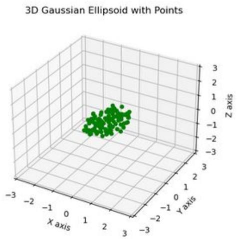
- Instead build custom renderer based on sampling the 3D scene









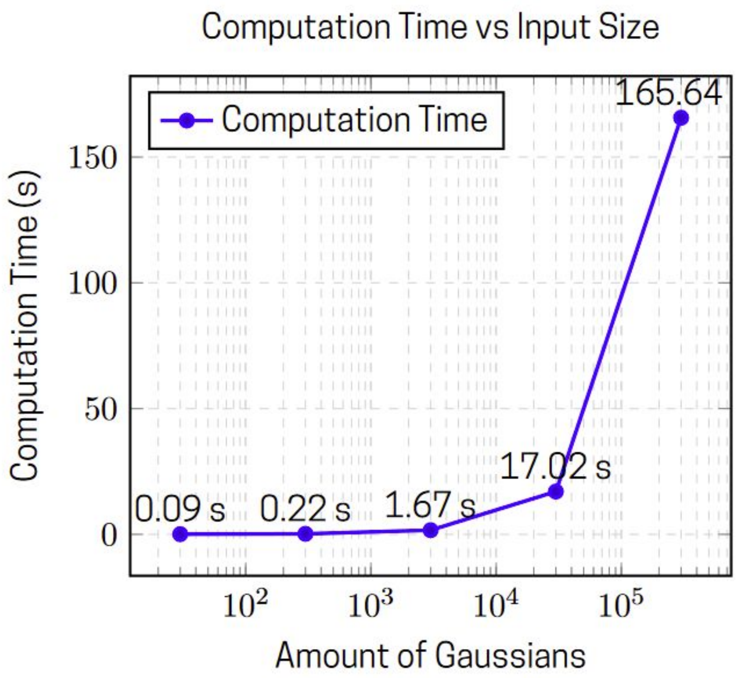


Rendering via Sampling

- ‘Sampling’ renderer was entirely written in pytorch and then tested for the computation of a single B-mode image

- performance was lacking

- runtime for a single 3D-GS run with low bound of 30k gaussians and 7k iterations with our sampler: ~10+ days



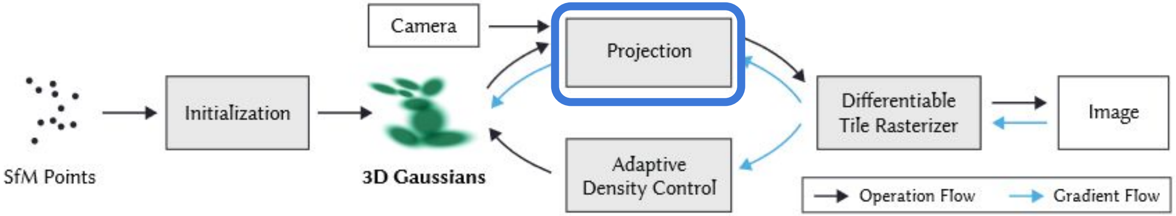
Computation time for 1 B-mode image of (1300 * 100 sample points) per amount of gaussians (range of 3 × 1e1 – 3 × 1e5 gaussians)

Overview so far

- custom sweeps for B-mode images in blender/IMFusion has good performance but results in a bad reconstruction

- ‘Sampling’ renderer performance undesirable but results look good

→ build a projection model that fits for our B-mode images as only viable option



Sources:
[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Trans. Graph., 42(4), 139-1.

Rendering via Projection

we define the following mathematical model for a orthographic projection matrix that should relate our real world scatterer and slice positions with gaussians in our model:

$$\mathbf{t}_1 = \text{random\_slice\_pos\_1}$$
$$\mathbf{t}_2 = \text{random\_slice\_pos\_2}$$

$$\mathbf{h} = \text{slice\_height}, \quad \mathbf{w} = \text{slice\_width}$$

$\rightarrow$

$$\text{top} = t_1[y], \quad \text{bottom} = t_1[y] - \mathbf{h}, \quad \text{right} = t_1[x], \quad \text{left} = t_1[x] - \mathbf{w}$$

$$\text{tanHalfFovX} = \frac{\text{right}}{\text{near\_plane}}, \quad \text{tanHalfFovY} = \frac{\text{top}}{\text{near\_plane}}$$

$$f_x = 2 \cdot \arctan(\text{tanHalfFovX}), \quad f_y = 2 \cdot \arctan(\text{tanHalfFovY})$$

$\rightarrow$

$$P_{\text{ortho}} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rendering via Projection

variables can be defined via tracked slices and their sizing data or are chose

$\mathbf{t}_1 = \text{random\_slice\_pos\_1}$

$\mathbf{t}_2 = \text{random\_slice\_pos\_2}$

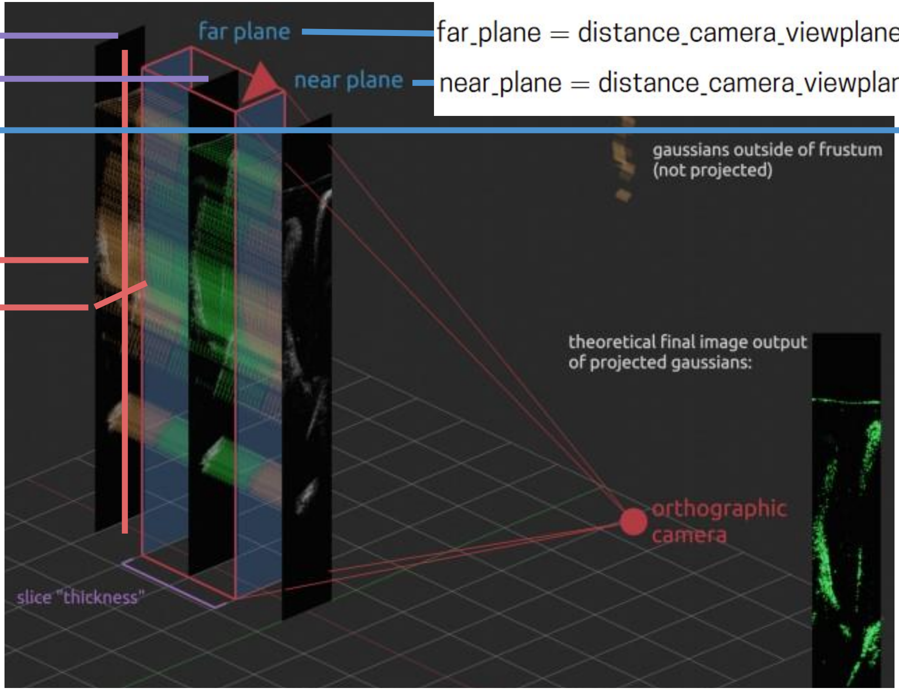
$d = \|\mathbf{t}_2 - \mathbf{t}_1\| \quad \approx \text{slice thickness}$

$\text{far\_plane} = \text{distance\_camera\_viewplane} + \frac{d}{2}$

$\text{near\_plane} = \text{distance\_camera\_viewplane} - \frac{d}{2}$

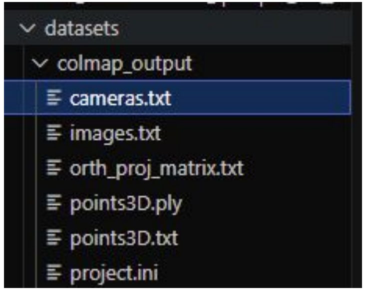
$\mathbf{h} = \text{slice\_height}$

$\mathbf{w} = \text{slice\_width}$

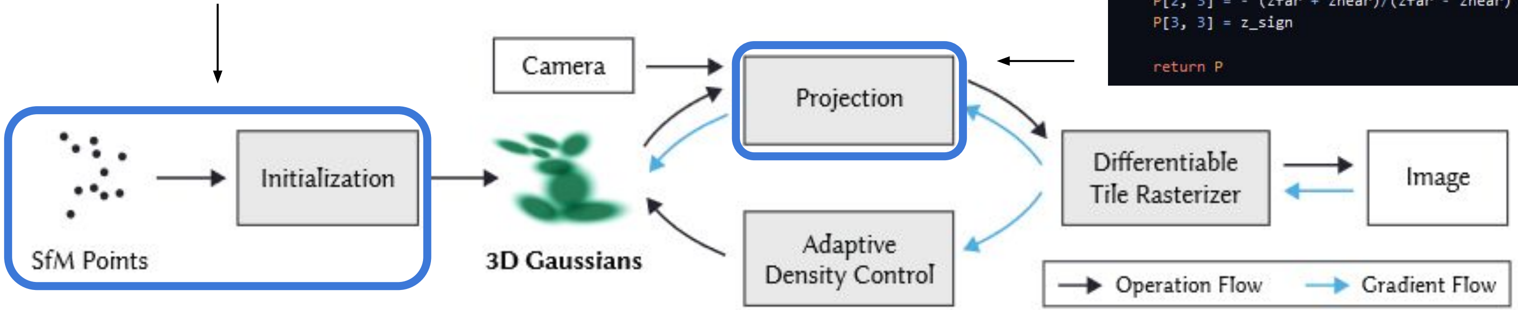


→ here we see a model of 3 slices, with a camera projecting all gaussians inside the frustum to the image on the right

Ultra Gaussian Splatting Pipeline




```python
def getProjectionMatrix(znear, zfar, fovX, fovY):
    tanHalfFovY = math.tan((fovY / 2))
    tanHalfFovX = math.tan((fovX / 2))

    top = tanHalfFovY * znear
    bottom = -top
    right = tanHalfFovX * znear
    left = -right

    P = torch.zeros(4, 4)

    z_sign = 1.0

    P[0, 0] = 2.0 / (right - left)
    P[0, 3] = - (right + left) / (right - left)
    P[1, 1] = 2.0 / (top - bottom)
    P[1, 3] = - (top + bottom) / (top - bottom)
    P[2, 2] = -2.0 / (zfar - znear)
    P[2, 3] = - (zfar + znear)/(zfar - znear)
    P[3, 3] = z_sign

    return P
```



Sources:
[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Trans. Graph., 42(4), 139-1.

Conclusion

- 3D Gaussian Splatting seems like a promising approach for optimizing 3D volumes in many modalities, especially medical ones
- modeling US is more difficult and typical approaches like creating additional sweeps, or a more realistic rendering model are not feasible or do not get quality results
- we have found no other gaussian splatting papers and ideas that seem directly beneficial to our current issues and use-cases
- specific orthographic projection for US in 3D-GS could give us the high-quality and high performance result we are looking for and using direct projection would only require minimal changes to the pipeline
- we have created a foundational mathematical model from which 3D-GS could be applied to US B-mode images directly in the future

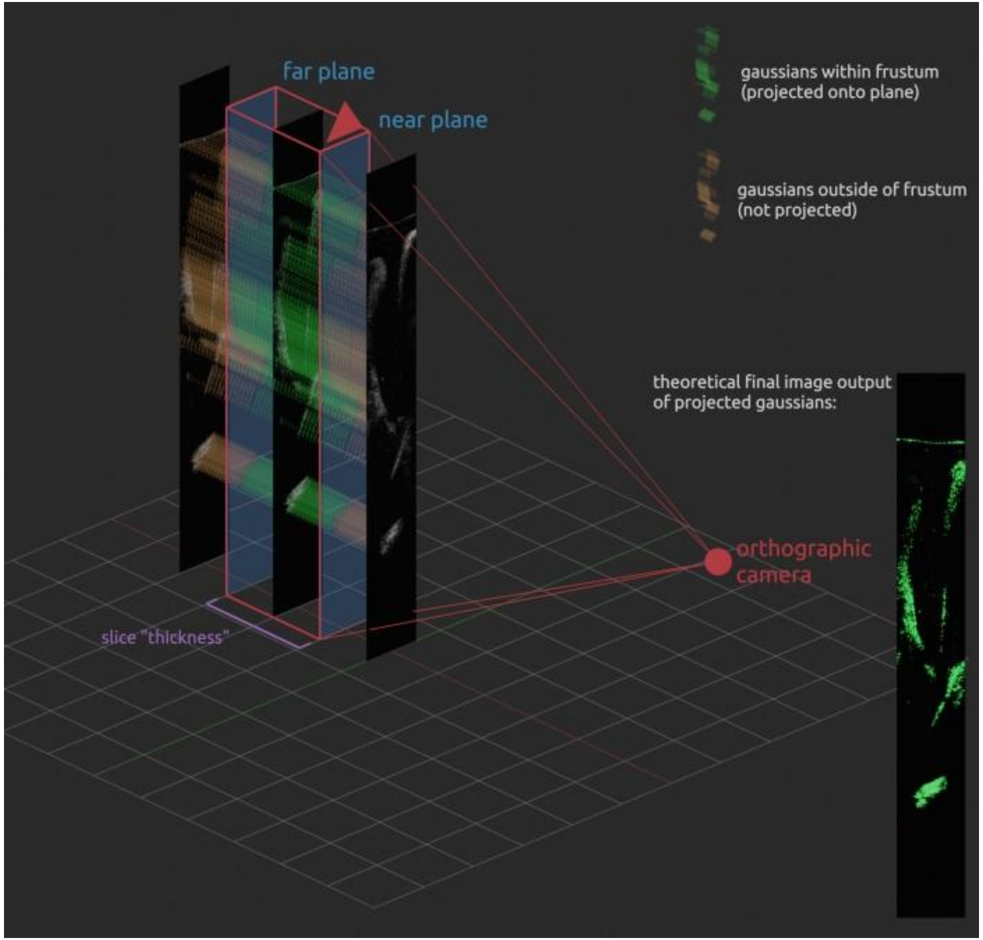
Thank you for your Attention!

Any questions?

Addendum: Rendering via Projection

→ we arrive at our final model for gaussian projection for a representation of scattering in our image data

→ here we see a model of 3 slices, with a camera projecting all gaussians inside the frustum to the image on the right



Addendum:
Choosing a camera position